Smart Contract Audit

cMATRA Surrender Pool Validator

Prepared for: Flux Point Studios

Date: April 14, 2026

Version: 1.1

Classification: Public

Language: Aiken 1.1.21 / Plutus V3 / stdlib 2.2.0

Scope: On-chain spend validator + off-chain transaction building

NO CRITICAL OR HIGH FINDINGS

1. Executive Summary

A line-by-line security audit was conducted on the cMATRA surrender pool validator — a parameterized Plutus V3 spend script written in Aiken that governs a pool of cMATRA tokens on the Cardano blockchain. The validator controls approximately 818 million cMATRA during a 6-month public redemption window.

The audit analyzed 15 distinct attack vectors covering the on-chain validator logic, the Aiken standard library functions it depends on, the off-chain transaction-building code, and Cardano-specific address manipulation attacks (frankenaddress). **No critical or high severity vulnerabilities were found.** Two low-severity findings in off-chain code were identified and remediated. The remaining findings are informational (design-level observations) or confirmed secure.

> **Overall Assessment:** The surrender pool validator is correctly implemented. Authorization and time-lock controls are enforced without exploitable gaps. The Aiken stdlib interval functions behave as expected with clean boundary semantics. The validator is suitable for mainnet deployment.

2. Validator Overview

2.1 Architecture

The surrender pool validator is a parameterized spend validator. Pool UTxOs at the script address hold cMATRA tokens with Void (unit) inline datums. The validator enforces two invariants for every pool UTxO spend:

1. **Authorization:** The designated administrator's verification key hash must appear in the transaction's required signers.
2. **Time control:** The transaction's validity range must fall entirely within the correct window — before the deadline for surrender processing, or after the deadline for pool withdrawal.

2.2 Parameters

Parameter	Type	Description
`admin_pkh`	Blake2b-224 hash (28 bytes)	Administrator verification key hash, baked in at compile time

`deadline`	POSIX milliseconds (Int)	End of the surrender window, baked in at compile time

2.3 Spending Paths

Redeemer	Conditions	Purpose
`ProcessSurrender`	Admin signed AND validity range entirely before deadline	Process a user's asset surrender during the open window
`AdminWithdraw`	Admin signed AND validity range entirely after deadline	Reclaim remaining pool balance after the window closes

2.4 Validator Source

```
validator surrender_pool(admin_pkh: VerificationKeyHash, deadline: Int) {
  spend(datum: Option<Void>, redeemer: SurrenderAction, _own_ref: OutputReference, tx:
Transaction) {
    expect Some(_) = datum
    let admin_signed = list.has(tx.extra_signatories, admin_pkh)

    when redeemer is {
      ProcessSurrender ->
        admin_signed && is_entirely_before(tx.validity_range, deadline)
      AdminWithdraw ->
        admin_signed && is_entirely_after(tx.validity_range, deadline)
    }
  }

  else(_) { fail }
}
```

3. Findings Summary

#	Severity	Finding	Location	Status
1	Low	Datum-less UTxO griefing in off-chain pool discovery	Off-chain	**Remediated**
2	Low	Frankenaddress: user address validation and configuration warnings	Off-chain	**Remediated**

3	**Design**	No on-chain value preservation (admin discretion by design)	On-chain	**Accepted**
4–15	**Secure**	Double satisfaction, deadline boundaries, unbounded ranges, authorization, datum robustness, multi-validator, reference scripts, parameters, PKH collision, concurrency, pool address frankenaddress, quarantine address frankenaddress	On-chain + Off-chain	**Verified**

4. Detailed Findings

Finding 1 — Datum-less UTxO Griefing [Low]

The on-chain validator correctly requires a datum via `expect Some(_) = datum`, making datum-less UTxOs unspendable. However, anyone can send cMATRA to the script address without attaching a datum. The off-chain pool discovery code did not check for datum presence, meaning it could select these poisoned UTxOs for transaction building, causing avoidable failures.

Remediation: Pool discovery now filters out UTxOs that lack both `inline_datum` and `data_hash`. Datum-less UTxOs are logged and skipped. The fix was applied to all three locations that query pool UTxOs: the core tool, the red-team suite, and the API service.

Finding 2 — No On-Chain Value Preservation [Design]

The validator does not enforce output constraints: it does not check that cMATRA is sent to the correct recipient, that legacy assets are routed to quarantine, or that the remaining pool balance is returned to the script address. The administrator has full discretion over how pool funds are routed in any transaction that passes the authorization and timing checks.

This is a deliberate design decision. Redemption rate enforcement occurs off-chain in the transaction-building service, which computes cMATRA amounts from a fixed rate table and constructs outputs accordingly. The validator's role is restricted to authorization and time-lock — it gates *who* can spend and *when*, while the off-chain service determines *how*.

> **Risk profile:** If the administrator signing key is compromised, the attacker could drain the entire pool by building arbitrary transactions. This risk is inherent to the architecture and is mitigated through operational security of the key and the host environment, rate limiting, and the API authentication layer added during the portal security audit.

5. Attack Vector Analysis

5.1 Double Satisfaction

SECURE

When multiple pool UTxOs are consumed in a single transaction, the validator executes once per input. Each invocation checks the same transaction context (same admin signature, same validity range). Since only the administrator can construct valid transactions, and modifying any transaction

output invalidates the administrator's signature, an attacker cannot exploit multi-input transactions to extract additional value.

5.2 Deadline Boundary Semantics

SECURE

The Aiken stdlib functions `is_entirely_before` and `is_entirely_after` were traced through their source code. The analysis confirms that there is a clean dead zone at the exact deadline boundary: a transaction whose validity range includes the deadline moment is rejected by *both* paths. There is no overlap and no gap between the two windows.

Validity Range	ProcessSurrender	AdminWithdraw
Upper bound = deadline − 1 (inclusive)	**Allowed**	Rejected
Upper bound = deadline (inclusive)	Rejected	Rejected
Lower bound = deadline (inclusive)	Rejected	Rejected
Lower bound = deadline + 1 (inclusive)	Rejected	**Allowed**
Unbounded (−inf, +inf)	Rejected	Rejected

5.3 Authorization Model

SECURE

The validator checks for `admin_pkh` in `tx.extra_signatories`. The Cardano ledger enforces that every public key hash listed in `required_signers` (which populates `extra_signatories`) has a corresponding verification key witness in the transaction. There is no mechanism to spoof this without possession of the administrator's private key.

5.4 Datum Robustness

SECURE

The `expect Some(_) = datum` pattern requires an inline datum to be present. UTxOs sent to the script address without a datum are unspendable by the validator. This is enforced at the Plutus level and cannot be bypassed.

5.5 Non-Spend Purpose Rejection

SECURE

The `else(_) { fail }` catch-all rejects any non-spend execution purpose (minting, certifying, withdrawing, voting, proposing). The validator's script hash cannot be leveraged to authorize operations outside of its intended scope.

5.6 Reference Script Substitution

SECURE

The script address is derived from the hash of the compiled bytecode, which includes the baked-in `admin_pkh` and `deadline` parameters. Substituting a different script or parameters would produce a different address. UTxOs at the correct address can only be spent by the correct script.

5.7 Parameter Immutability

SECURE

Parameters are applied at compile time via `aiken blueprint apply` and embedded in the CBOR bytecode. They cannot be altered post-deployment. Changing parameters generates a new script with a new hash and address.

5.8 Concurrency

ACCEPTABLE

The eUTxO model ensures that each pool UTxO can only be consumed by one transaction. Concurrent users targeting the same UTxO will see one succeed and the other receive a retryable error. For higher throughput, the pool can be distributed across multiple UTxOs (the off-chain code already supports multi-UTxO selection).

6. Frankenaddress Analysis

A *frankenaddress* (sometimes called a Frankenstein address) is a Cardano address that combines one entity's payment credential with a different entity's staking credential. On Cardano, addresses consist of two parts: a payment credential (who can spend) and an optional staking credential (who earns delegation rewards). By constructing an address with a script's payment hash but an attacker's staking key, the attacker could earn staking rewards on ADA locked at that address without being able to spend it.

Three address surfaces were analyzed for frankenaddress exposure:

6.1 Pool / Script Address

SECURE

The surrender pool script address is constructed as an enterprise-type address with only a payment credential (the script hash) and no staking credential. An attacker could construct an alternative address using the same script hash combined with their staking key. However, the off-chain pool discovery code queries only the canonical configured address. UTxOs at a frankenaddress variant would never be discovered, selected, or spent. Additionally, when the transaction builder returns remaining pool balance to the script, it always routes to the canonical address — never to a frankenaddress variant.

6.2 User Address

REMEDIATED

The transaction builder sends cMATRA to the user's provided address. An attacker could theoretically provide a script-payment address (where the payment credential is a script hash rather than a verification key), causing cMATRA to be sent to an address the user cannot spend from.

Remediation: The transaction-building endpoint now validates that the user's address has a verification key payment credential (not a script hash). Requests with script-payment addresses are rejected. This ensures cMATRA is always delivered to a key-controlled wallet that the user can spend from.

6.3 Quarantine Address

REMEDIATED

Legacy assets surrendered by users are routed to a quarantine address where they are permanently removed from circulation. If this address were configured with a staking credential, whoever controls that staking key would earn delegation rewards on the minimum ADA locked alongside the quarantined assets.

Remediation: The service now validates the quarantine address at startup and logs a warning if a staking credential is present. The operational recommendation is to use an enterprise-type address (no staking component) for the quarantine, ensuring no party can extract staking rewards from burned asset UTxOs.

> **Deployment guidance:** Both the script address and the quarantine address should be enterprise-type addresses with no staking credential. This is the default when generating addresses from a script hash alone. The service validates this at startup and warns if a staking component is detected.

7. Test Coverage

Source	Tests	Coverage
On-chain	7	Admin ProcessSurrender, post-deadline ProcessSurrender (fail), pre-deadline AdminWithdraw (fail), post-deadline AdminWithdraw, non-admin ProcessSurrender (fail), non-admin AdminWithdraw (fail), missing datum (fail)
Red-team	8	All on-chain tests + wrong redeemer data (fail), fabricated UTxO (fail), admin happy-path ProcessSurrender, admin happy-path AdminWithdraw. Executed against live preprod deployment.
Preprod	9 stages	Full lifecycle: wallet creation, cMATRA minting, pool deployment, user surrender, red-team attacks, admin withdrawal. 9/9 stages passed.

8. Conclusion

The cMATRA surrender pool validator is a minimal, well-scoped Plutus V3 script that correctly enforces its two design invariants: administrator authorization and time-locked spending windows. The Aiken stdlib interval functions provide clean boundary semantics with no exploitable edge cases at the deadline.

Two low-severity findings were identified and remediated: datum-less UTxO griefing in pool discovery, and frankenaddress exposure across three address surfaces (pool, user, quarantine). The validator's deliberate omission of on-chain value preservation is documented as a design decision with appropriate operational mitigations.

> **Assessment:** The surrender pool validator is correctly implemented and suitable for mainnet deployment. No critical, high, or medium severity vulnerabilities were identified.
